Teori *Network Analysis*

Capaian pembelajaran

- Mahasiswa memahami maksud dari teori *network analysis* ;
- Mahasiswa memahami cara melakukan analisis algoritma pada teori *network analysis*

Agenda

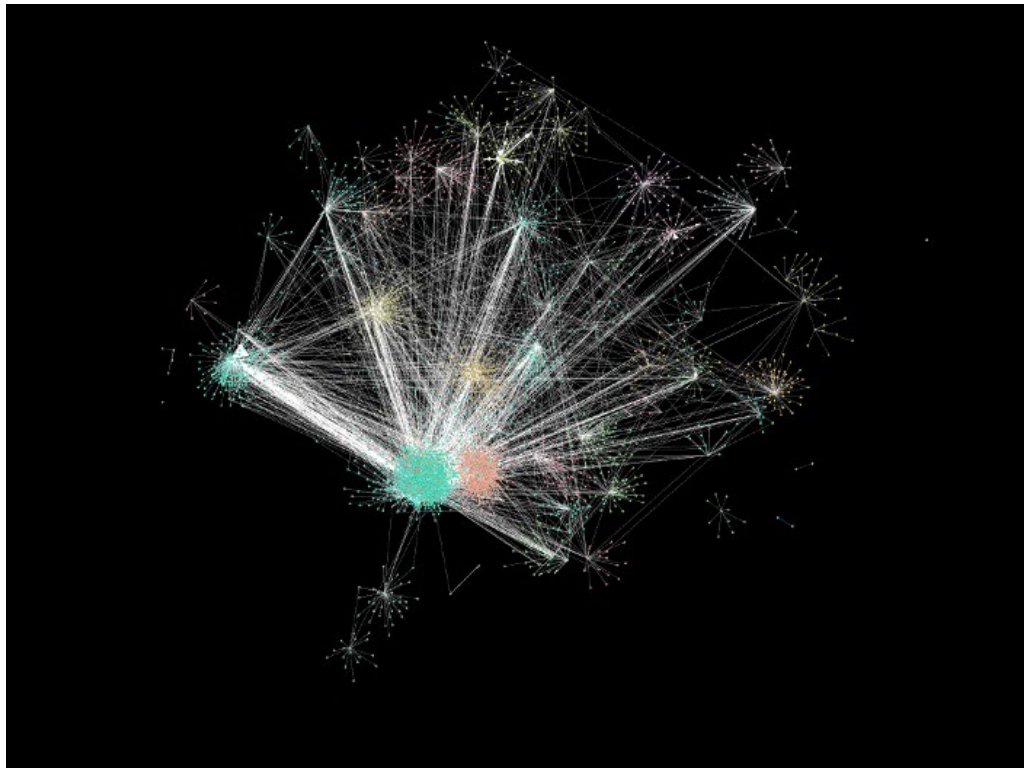
- Pengenalan Teori *Network Analysis*
- *Shortest Path*
- Pengukuran *Centrality*

Pengenalan Teori *Network Analysis*

- Kita tahu bahwa data yang saling berhubungan dapat direpresentasikan sebagai jaringan. Dalam teori analisis jaringan, kami mempelajari detail metodologi yang dikembangkan untuk mengeksplorasi dan menganalisis data yang direpresentasikan sebagai jaringan . Mari kita lihat beberapa aspek penting dari teori analisis jaringan di bagian ini.
- Pertama, perhatikan bahwa simpul dalam jaringan bertindak sebagai unit dasar. Jaringan adalah jaringan simpul yang saling terhubung di mana setiap koneksi mewakili hubungan antara berbagai entitas yang sedang diselidiki. Penting untuk mengukur kegunaan dan pentingnya sebuah simpul dalam jaringan dalam konteks masalah yang kita coba pecahkan. Ada berbagai teknik yang dapat membantu kami mengukur pentingnya.

Pengenalan Teori *Network Analysis*

- “Community Detection of ISIS Twitter Accounts”.



Shortest Path

- Path adalah urutan node antara node awal dan node akhir, di mana tidak ada node yang muncul dua kali pada jalur tersebut. Jalur mewakili rute antara titik awal dan akhir yang dipilih. Ini akan menjadi himpunan simpul, p, menghubungkan simpul awal dengan simpul akhir. Tidak ada simpul yang diulang di hal.
- Length of path dihitung dengan menghitung sisi-sisi penyusunnya. Dari semua opsi, jalur dengan panjang terkecil disebut jalur terpendek. Penghitungan jalur terpendek digunakan secara luas dalam algoritme teori graf, tetapi tidak selalu mudah untuk dihitung. Ada berbagai algoritme yang dapat digunakan untuk menemukan jalur terpendek antara node awal dan node akhir. Salah satu algoritme paling populer yang dapat digunakan untuk menemukan jarak terpendek antara simpul awal dan simpul akhir adalah algoritme Dijkstra, yang diterbitkan pada akhir 1950-an. Ini menghitung jalur terpendek dalam grafik.
- Ini dapat digunakan oleh perangkat Global Positioning System (GPS) untuk menghitung jarak minimum antara sumber dan tujuan. Algoritme Dijkstra juga digunakan dalam algoritme perutean jaringan.

Shortest Path

- Membuat Neighborhood: Menemukan strategi untuk membuat lingkungan di sekitar simpul yang diinginkan sangat penting untuk algoritme grafik. Metodologi untuk menciptakan lingkungan didasarkan pada pemilihan asosiasi langsung dengan simpul yang diinginkan. Salah satu cara untuk menciptakan lingkungan adalah dengan memilih strategi k-order yang memilih simpul yang k hop jauhnya dari simpul yang diinginkan.
- Triangles: Dalam teori graf, menemukan simpul yang terhubung dengan baik satu sama lain penting untuk tujuan analisis. Salah satu tekniknya adalah mencoba mengidentifikasi segitiga, yang merupakan subgraf yang terdiri dari tiga simpul yang terhubung langsung satu sama lain, dalam jaringan. Jika egonet dari sebuah simpul, m, terdiri dari tiga simpul, termasuk simpul m, maka egonet ini adalah segitiga. Simpul m akan menjadi ego dan dua simpul yang terhubung akan menjadi alter, katakanlah simpul A dan simpul B. Jika kedua alter diketahui sebagai kasus penipuan, kita dapat dengan aman mendeklarasikan simpul m sebagai penipuan juga. Jika salah satu perubahan tersebut terlibat dalam penipuan, kami tidak dapat memberikan bukti yang meyakinkan, tetapi kami perlu melakukan penelusuran lebih lanjut atas bukti penipuan tersebut.

SHORTEST PATH

- Density: Kami menamakan graf di mana setiap simpul terhubung langsung ke setiap simpul lainnya sebagai jaringan yang terhubung penuh. Jika kita memiliki jaringan yang sepenuhnya terhubung, N, maka jumlah tepi dalam jaringan dapat diwakili oleh yang berikut:

$$Edges_{total} = \binom{N}{2} = \frac{N(N-1)}{2}$$

- Sekarang, di sinilah kepadatan berperan. Density mengukur jumlah edge yang diamati hingga jumlah maksimum edge, jika EdgesObserved adalah jumlah edge yang ingin kita amati. Itu dapat dibingkai sebagai berikut:

$$density = \frac{Edges_{observed}}{Edges_{total}}$$

Pengukuran *Centrality*

Ada ukuran yang berbeda untuk memahami sentralitas simpul tertentu dalam graf atau subgraf. Ukuran sentralitas berikut ini banyak digunakan dalam analisis grafik:

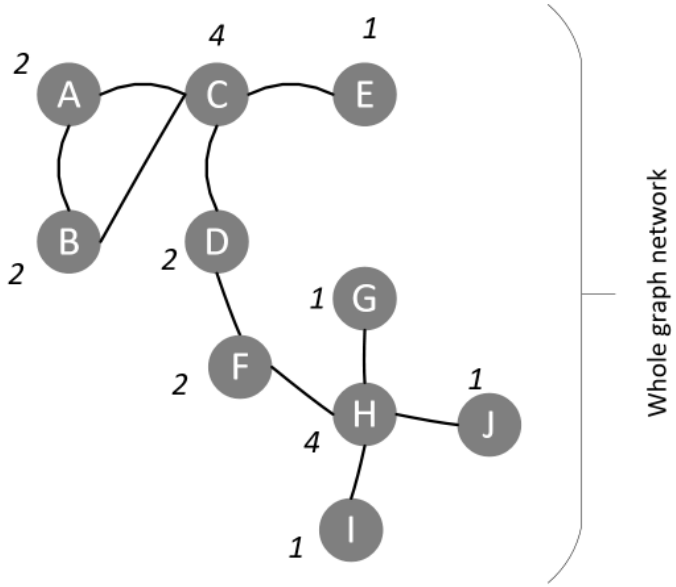
- Degree
- Betweenness
- Closeness
- Eigenvector

PEGUKURAN *CENTRALITY: Degree*

- Banyaknya sisi yang terhubung ke simpul tertentu disebut derajatnya. Ini dapat menunjukkan seberapa baik simpul tertentu terhubung dan kemampuannya untuk menyebarkan pesan dengan cepat ke seluruh jaringan.
- Let's consider aGraph = ($\mathcal{V}$, $\mathcal{E}$), where $\mathcal{V}$ represents a set of vertices and $\mathcal{E}$ represents a set of edges. Recall that aGraph has $|\mathcal{V}|$ vertices and $|\mathcal{E}|$ edges. If we divide the degree of a node by ($|\mathcal{V}|$ -1), it is called degree centrality:

$$C_{DC_a} = \frac{deg(a)}{|\mathcal{V}| - 1}$$

$$C_{DC_c} = \frac{deg(c)}{|\mathcal{V}| - 1} = \frac{4}{10 - 1} = 0.44$$



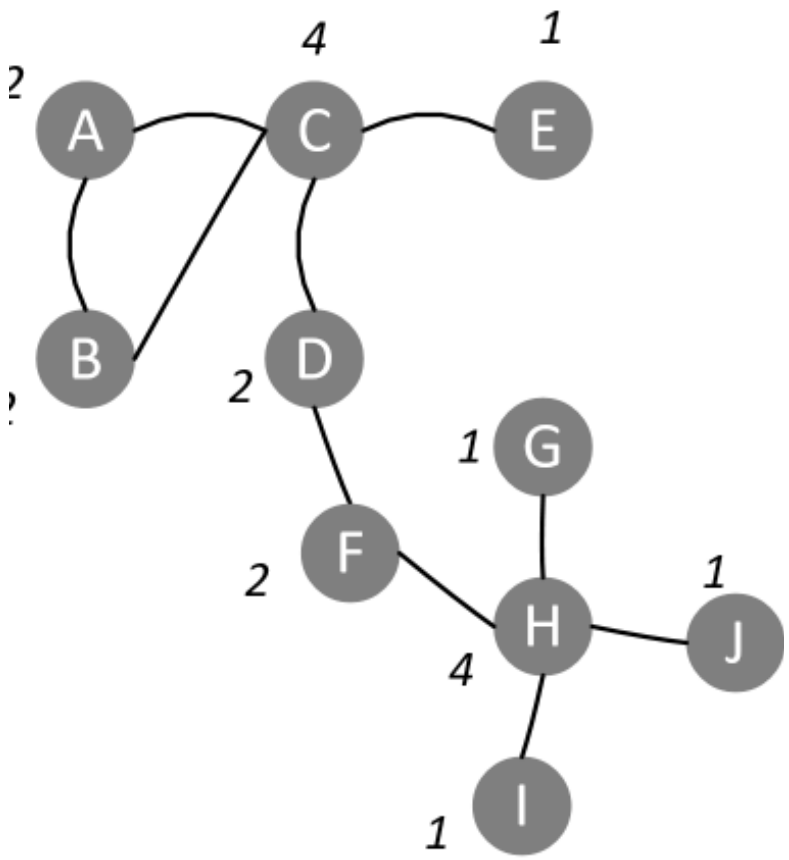
PEGUKURAN *CENTRALITY: betweenness*

- Betweenness adalah ukuran sentralitas dalam grafik. Dalam konteks media sosial, ini akan mengukur probabilitas bahwa seseorang menjadi bagian dari komunikasi dalam subkelompok. Untuk jaringan komputer, keantaraan akan mengukur efek negatif pada komunikasi antara node grafik, jika terjadi kegagalan titik.
- To calculate the betweenness of vertex a in a certain aGraph = ($\mathcal{V}$, $\mathcal{E}$), follow these steps:
  - Compute the shortest paths between each pair of vertices in aGraph. Let's represent this with Nshortest Total.
  - From Nshortest Total, count the number of shortest paths that pass through vertex a. Let's represent this with Nshortest a.
  - Calculate the betweenness with

$$C_{betweenness_a} = \frac{n_{shortest_a}}{n_{shortest_{Total}}}$$

PEGUKURAN *CENTRALITY: Closeness*

- Let's take a graph, g. The fairness of vertex a in graph g is defined as the sum of vertex a's distance from other vertices. Note that the centrality of a particular vertex quantifies its total distance from all the other vertices.
- The opposite of fairness is closeness.



PENGUKURAN *CENTRALITY: Eigenvector*

PEGUKURAN CENTRALITY: Eigenvector

- **Install NetworkX dengan menggunakan command berikut:**

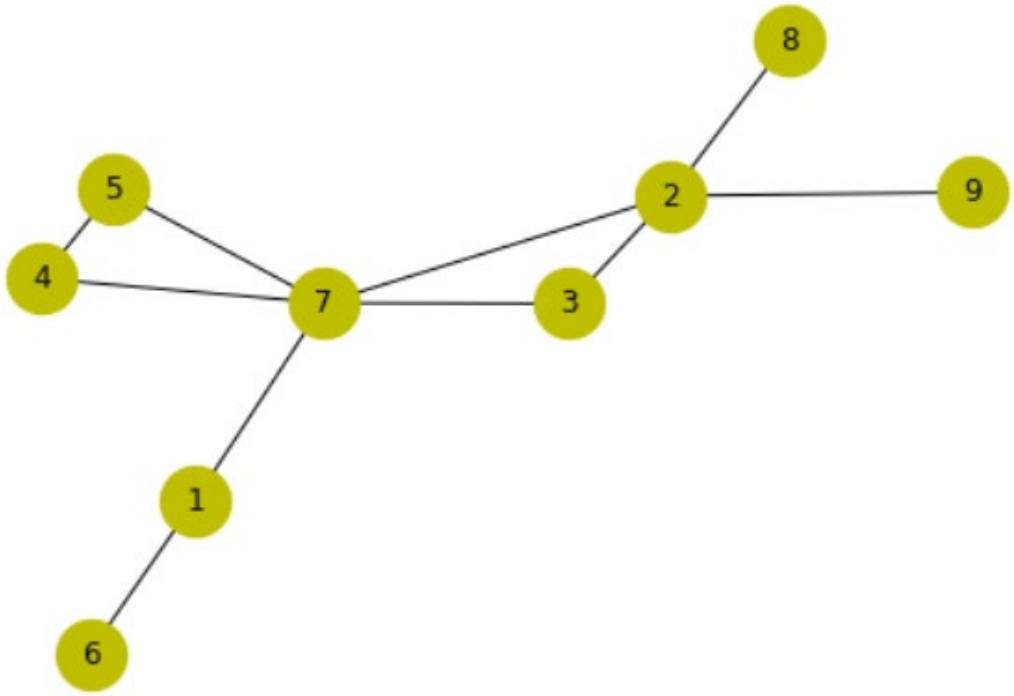
```
pip install networkx
```

- Atau dengan command berikut

```
conda install -c anaconda networkx
```

PENGUKURAN *CENTRALITY: EIGENVECTOR*

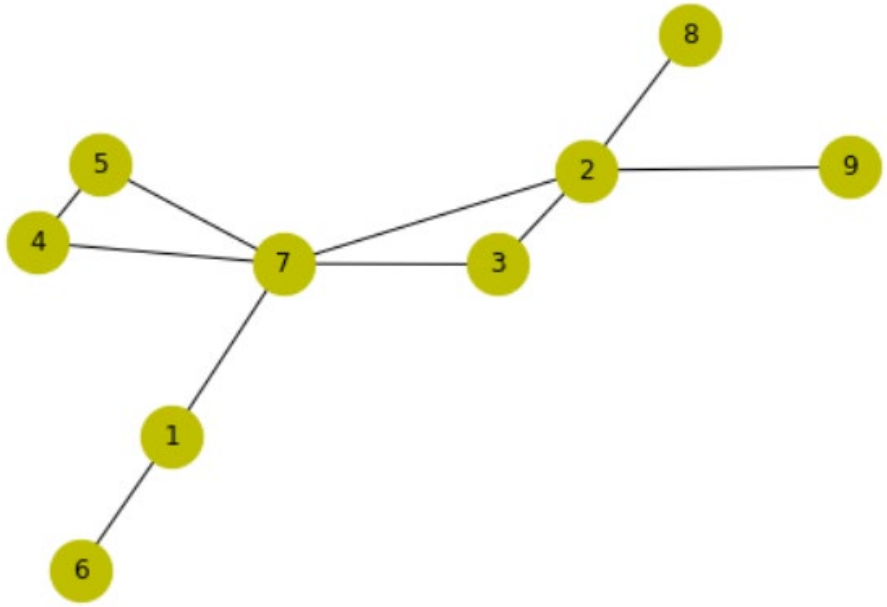
- Let's create a network and then try to calculate its centrality metrics. The following code block illustrates this:




```
import networkx as nx
import matplotlib.pyplot as plt
vertices = range(1,10)
edges = [(7,2), (2,3), (7,4), (4,5), (7,3), (7,5),(1,6),(1,7),(2,8),(2,9)]
G = nx.Graph()
G.add_nodes_from(vertices)
G.add_edges_from(edges)
nx.draw(G, with_labels=True,node_color='y',node_size=800)
```

PENGUKURAN *DEGREE CENTRALITY*

- Degree centrality is a measure of the number of connections a particular node has in the network. The people most popular or more liked usually are the ones who have more friends.




```
nx.degree_centrality(G)
```

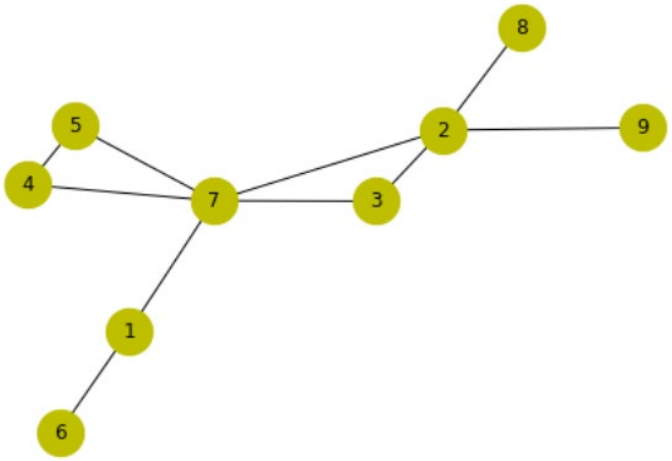
```
{1: 0.25,
 2: 0.5,
 3: 0.25,
 4: 0.25,
 5: 0.25,
 6: 0.125,
 7: 0.625,
 8: 0.125,
 9: 0.125}
```

Pengukuran betweenness centrality

- The Betweenness Centrality is the centrality of control. It represents the frequency at which a point occurs on the geodesic (shortest paths) that connected pair of points. It quantifies how many times a particular node comes in the shortest chosen path between two other nodes.

```
nx.betweenness_centrality(G)
```

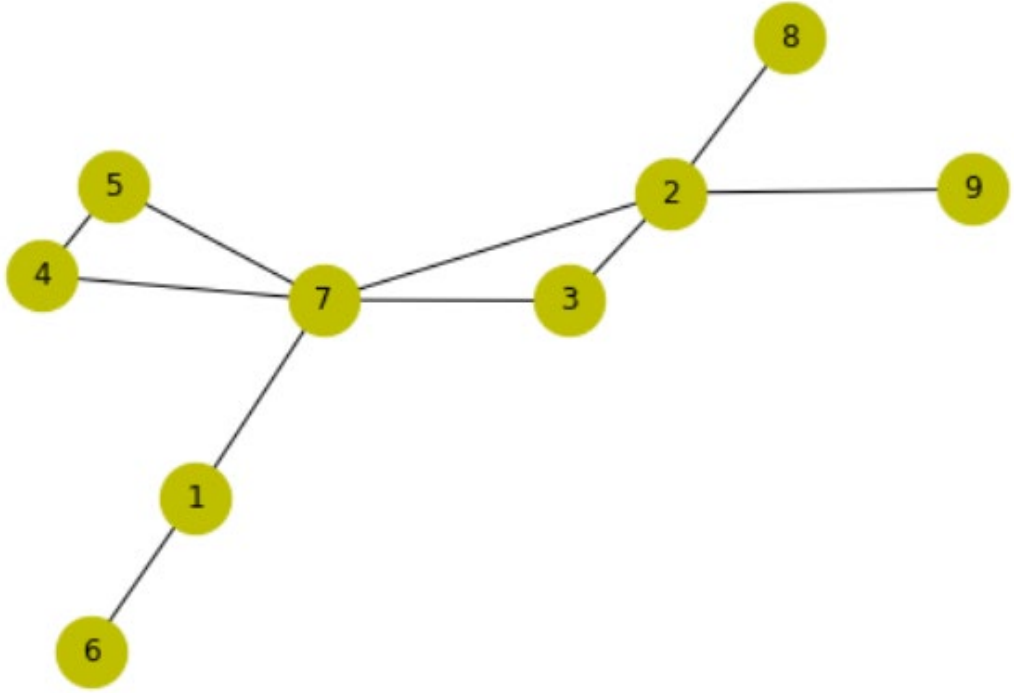
```
{1: 0.25,
 2: 0.4642857142857142 5,
 3: 0.0,
 4: 0.0,
 5: 0.0,
 6: 0.0,
 7: 0.7142857142857142,
 8: 0.0,
 9: 0.0}
```



Pengukuran closeness centrality

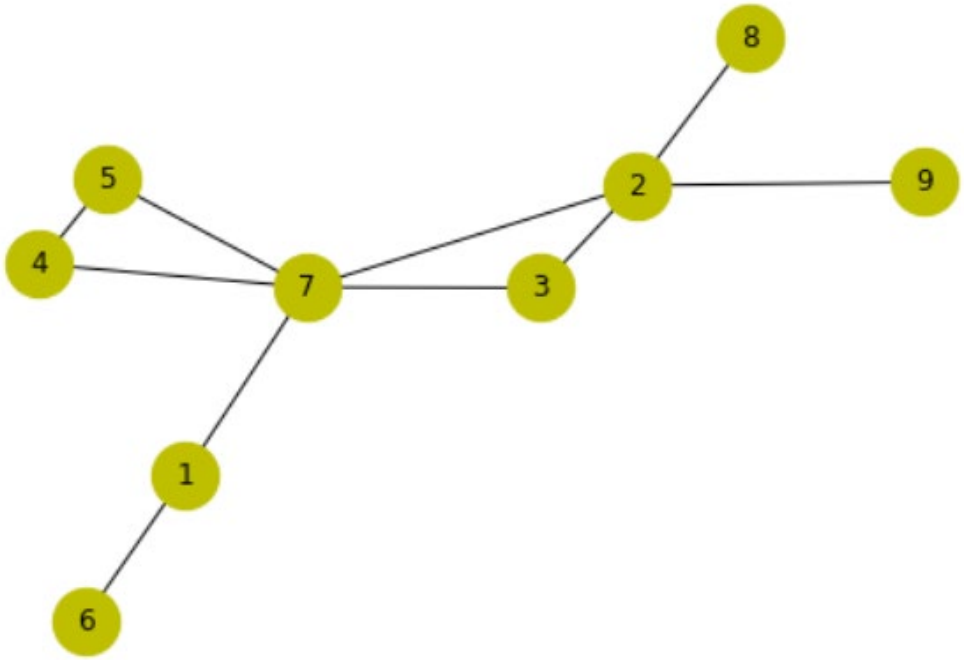
```
nx.closeness_centrality(G)
```

```
{1: 0.5,
 2: 0.6153846153846154,
 3: 0.5333333333333333,
 4: 0.4705882352941176 4,
 5: 0.4705882352941176 4,
 6: 0.3478260869565217 3,
 7: 0.7272727272727273,
 8: 0.4,
 9: 0.4}
```



Pengukuran eigenvector centrality

- Eigenvector memberikan nilai kepada semua simpul dalam suatu grafik dengan mengukur tingkat kepentingannya dalam suatu network



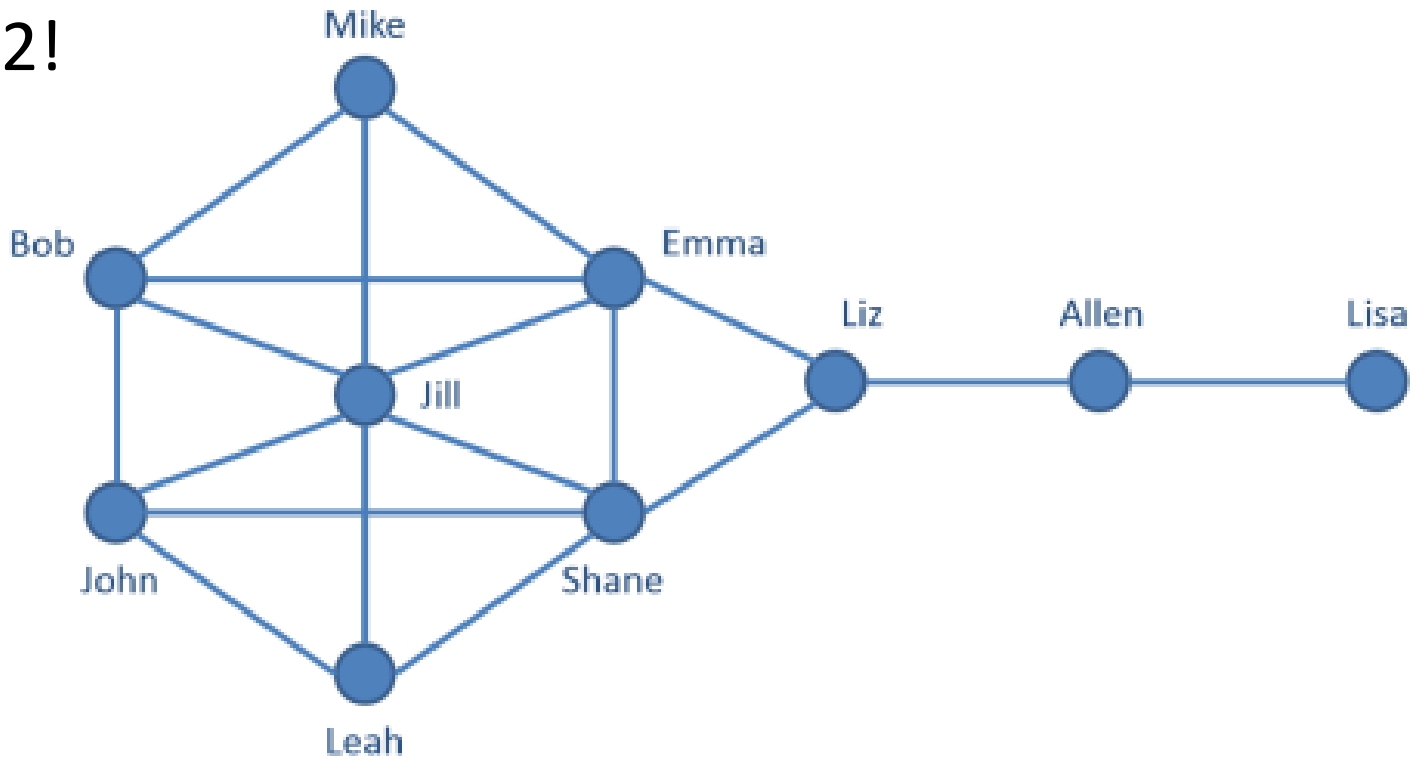

```
centrality = nx.eigenvector_centrality(G)
```

```
sorted((v, '{:0.2f}'.format(c)) for v, c in centrality.items())
```

```
[(1, '0.24'),
 (2, '0.45'),
 (3, '0.36'),
 (4, '0.32'),
 (5, '0.32'),
 (6, '0.08'),
 (7, '0.59'),
 (8, '0.16'),
 (9, '0.16')]
```

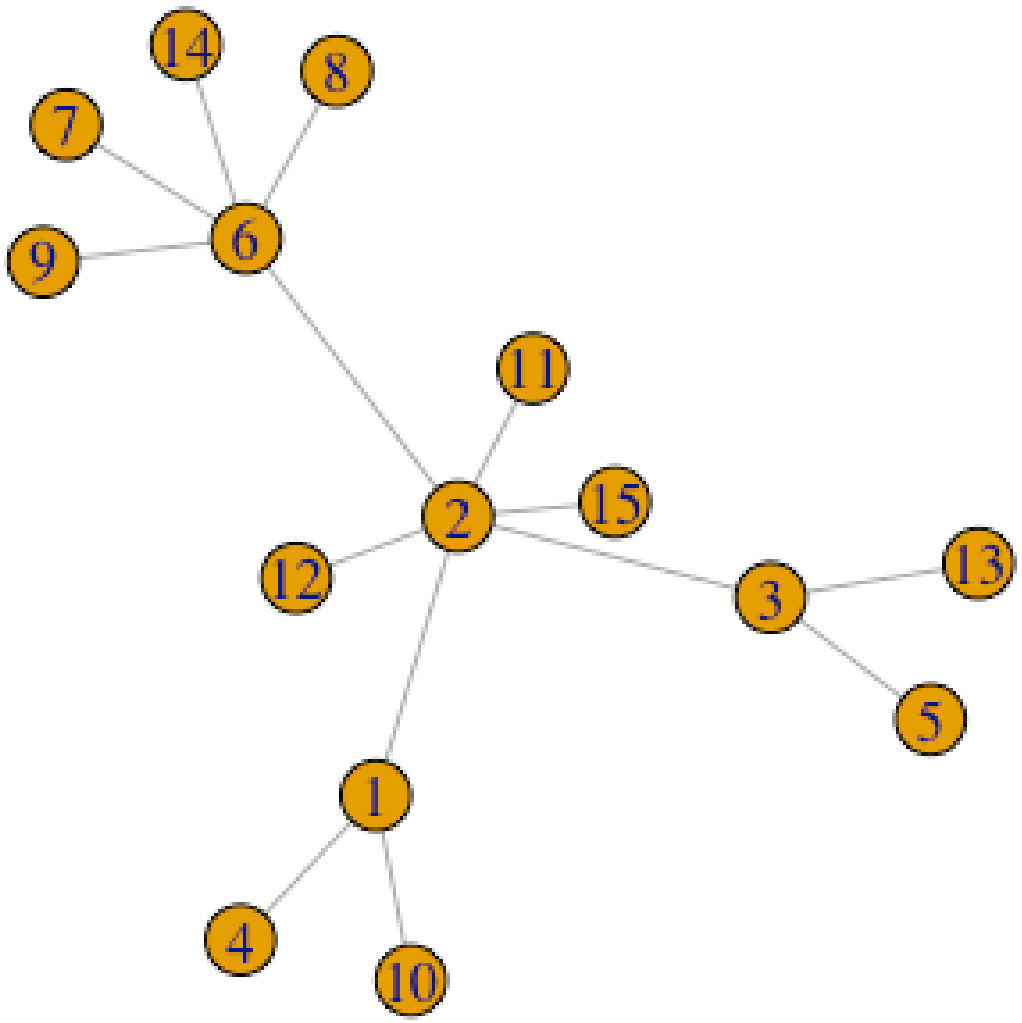
LATIHAN

- Buat network
- Hubungkan sebagai berikut:
- Hitung eigenvector masing2!



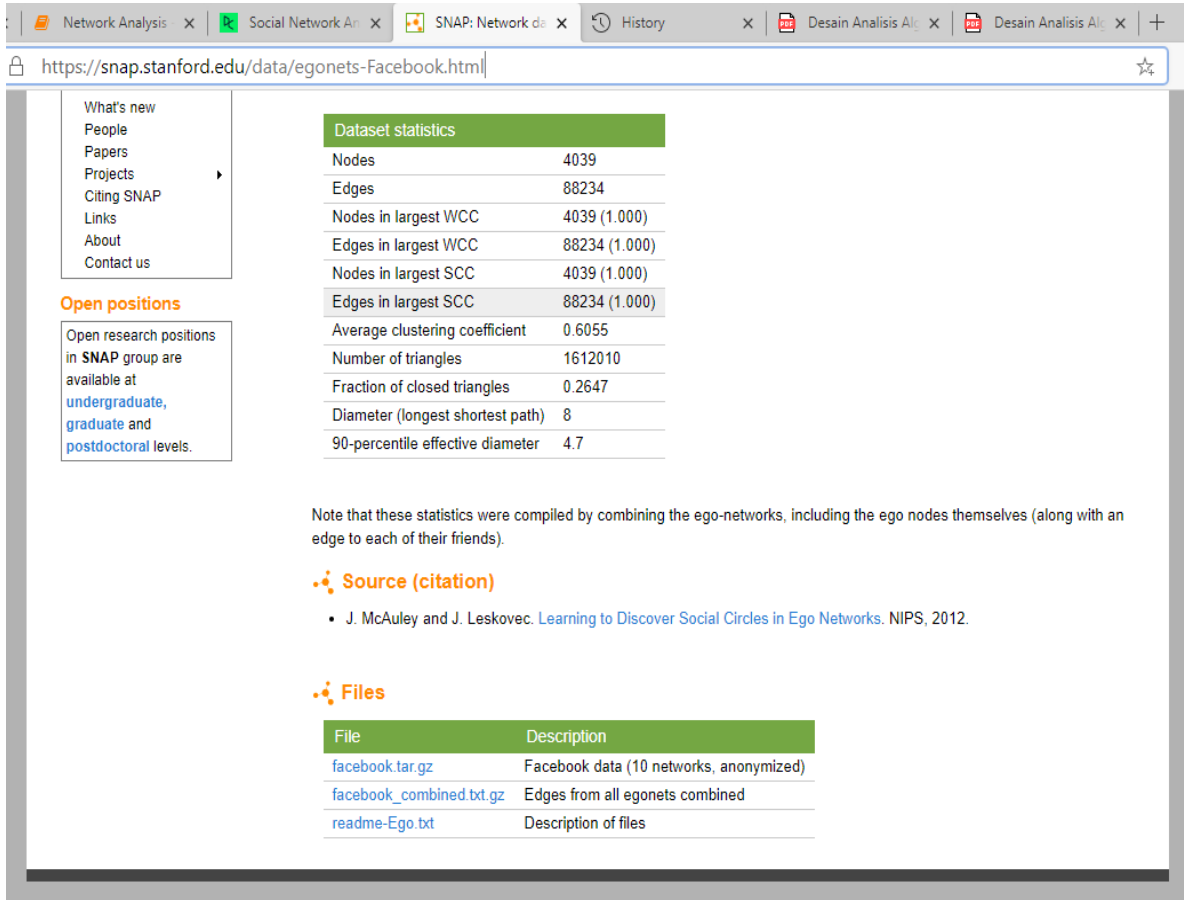
LATIHAN

- Buat network
- Hubungkan sebagai berikut:
- Hitung eigenvector!



Membuat Network berdasarkan data pemodelan facebook dari Stanford

- Download data dari https://snap.stanford.edu/data/egonets-Facebook.html



Membuat Network berdasarkan data pemodelan facebook dari Stanford

- Muat data untuk dibentuk edge list nya

```
G_fb = nx.read_edgelist("facebook_combined.txt", create_using = nx.Graph(), nodetype=int)
```

- “content/drive/MyDrive/facebook_combined.txt”

Membuat Network berdasarkan data pemodelan facebook dari Stanford

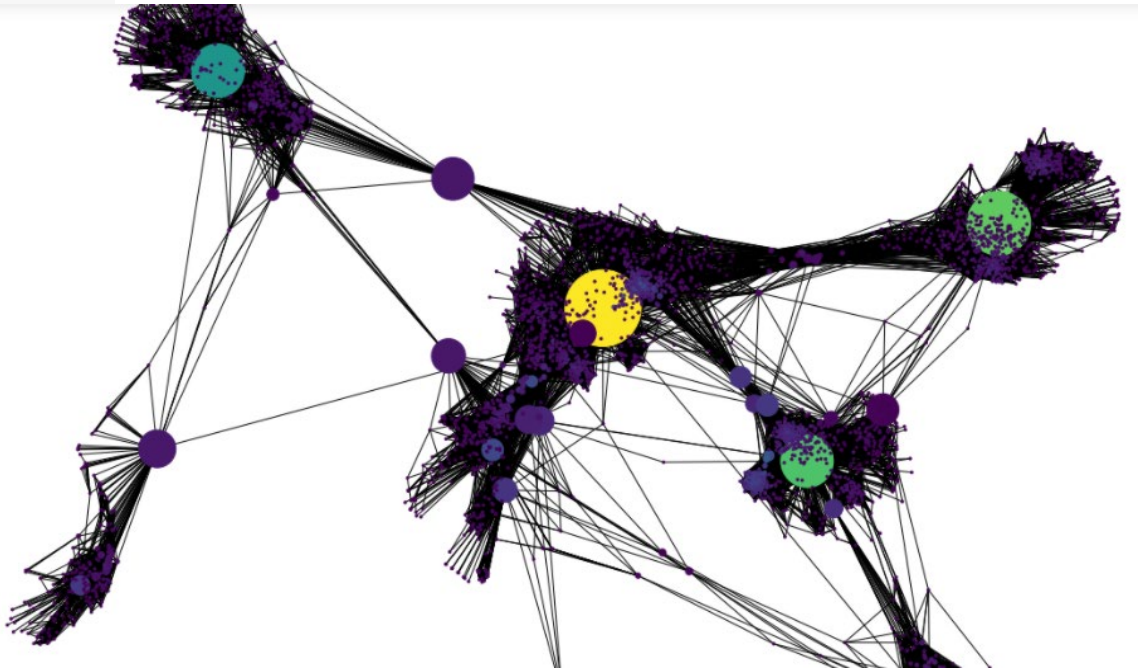
- Print informasi jumlah nodes dan edges

```
print(nx.info(G_fb))
```

```
Name:
Type: Graph
Number of nodes: 4039
Number of edges: 88234
Average degree:  43.6910
```

Membuat Network berdasarkan data pemodelan facebook dari Stanford

- Visualisasi network berdasarkan degree dan betweenness centrality

```
pos = nx.spring_layout(G_fb)
betCent = nx.betweenness_centrality(G_fb, normalized=True, endpoints=True)
node_color = [20000.0 * G_fb.degree(v) for v in G_fb]
node_size =  [v * 10000 for v in betCent.values()]
plt.figure(figsize=(20,20))
nx.draw_networkx(G_fb, pos=pos, with_labels=False,
                 node_color=node_color,
                 node_size=node_size )
plt.axis('off')
```

```
(-0.9338846564292907,
 0.6786860585212707,
 -1.0758122205734253,
 0.5920566320419312)
```